El hilo: el modelo, la configuración y la línea base

Asesoría del proyecto final — Semana 1

Carlos Orrego

Razonamiento Automático — Prof. Fabián Serrano
Universidad Nacional de Colombia — Sede Manizales

5 de noviembre de 2026

Hoy, en una frase

Al salir de esta clase. . .

. . . cada uno tiene un modelo de lenguaje respondiendo desde Python y sabe medir qué tan mal demuestra *sin ayuda*.

Ese «qué tan mal» tiene nombre: la línea base.

Recorrido

Minutos	Parte
0–10	1. Por qué: el modelo propone, Isabelle decide
10–25	2. El proyecto en un mapa
25–35	3. Modelos gratuitos
35–45	4. El kit del profesor Fabián
45–70	5. Taller en vivo
70–80	6. Temas y tarea

Los cuatro jueves

Jueves	Nombre	Construimos
1 (hoy)	El hilo	el modelo y la línea base
2	La biblioteca	buscar en los documentos y citar
3	La flecha roja	verificar con Isabelle y corregir
4	El espejo	evaluar, analizar errores, informe

Cada jueves deja algo funcionando. Nada se tira: lo de hoy se reutiliza en las tres semanas siguientes.

Pregunta para arrancar

Le pido a un chatbot una demostración.

Me la da, bien escrita, segura de sí misma.

¿Cómo sé que es correcta?

Un modelo de lenguaje escribe; no verifica

- Un modelo de lenguaje predice la siguiente palabra más probable.

Un modelo de lenguaje escribe; no verifica

- Un modelo de lenguaje predice la siguiente palabra más probable.
- Produce texto plausible, no necesariamente verdadero.

Un modelo de lenguaje escribe; no verifica

- Un modelo de lenguaje predice la siguiente palabra más probable.
- Produce texto plausible, no necesariamente verdadero.
- Cuando afirma algo falso con total seguridad, se llama alucinación.

Un modelo de lenguaje escribe; no verifica

- Un modelo de lenguaje predice la siguiente palabra más probable.
- Produce texto plausible, no necesariamente verdadero.
- Cuando afirma algo falso con total seguridad, se llama alucinación.
- En matemáticas, una alucinación es una demostración que *parece* correcta y no lo es, o una cita a un libro que no dice eso.

Isabelle hace justo lo contrario

El modelo

- escribe con fluidez
- se equivoca con seguridad
- no sabe que se equivocó

Isabelle

- no se deja convencer por el estilo
- acepta o rechaza
- dice *dónde* falla

Isabelle hace justo lo contrario

El modelo

- escribe con fluidez
- se equivoca con seguridad
- no sabe que se equivocó

Isabelle

- no se deja convencer por el estilo
- acepta o rechaza
- dice *dónde* falla

El modelo propone; Isabelle decide.

Un ejemplo real: el ejercicio 1 del benchmark

El enunciado:

```
lemma ejercicio_1:
  assumes 1: "p ⟶ q" and
          2: "p"
  shows "q"
```

Un ejemplo real: el ejercicio 1 del benchmark

El enunciado:

```
lemma ejercicio_1:
  assumes 1: "p ⟶ q" and
          2: "p"
  shows "q"
```

Lo que propone un modelo:

```
proof -
  show "q" using 2 1 by (rule mp)
qed
```

Un ejemplo real: el ejercicio 1 del benchmark

El enunciado:

```
lemma ejercicio_1:
  assumes 1: "p ⟶ q" and
          2: "p"
  shows "q"
```

Lo que propone un modelo:

```
proof -
  show "q" using 2 1 by (rule mp)
qed
```

¿Ven el problema? Piénsenlo diez segundos antes de pasar.

Lo que responde Isabelle

Isabelle

```
Failed to apply initial proof method
```

Lo que responde Isabelle

Isabelle

```
Failed to apply initial proof method
```

- La regla `mp` espera los hechos en un orden: *primero* la implicación $p \longrightarrow q$, *luego* la premisa p.
- El modelo los pasó al revés: `using 2 1`.

Lo que responde Isabelle

Isabelle

```
Failed to apply initial proof method
```

- La regla `mp` espera los hechos en un orden: *primero* la implicación $p \longrightarrow q$, *luego* la premisa p.
- El modelo los pasó al revés: `using 2 1`.

Corregido:

```
show "q" using 1 2 by (rule mp)
```

Se ven casi iguales. Solo una es una demostración.

Qué es un RAG, sin tecnicismos

RAG = *Retrieval-Augmented Generation*: generación aumentada con recuperación.

Qué es un RAG, sin tecnicismos

RAG = *Retrieval-Augmented Generation*: generación aumentada con recuperación.

La analogía

Un estudiante en un examen a libro abierto:

1. lee la pregunta,
2. busca en el libro las páginas útiles,
3. responde apoyándose en ellas, y dice de qué página lo sacó.

Qué es un RAG, sin tecnicismos

RAG = *Retrieval-Augmented Generation*: generación aumentada con recuperación.

La analogía

Un estudiante en un examen a libro abierto:

1. lee la pregunta,
2. busca en el libro las páginas útiles,
3. responde apoyándose en ellas, y dice de qué página lo sacó.

Sin el paso 2, el modelo responde «de memoria»: ahí es donde más inventa.

El proyecto tiene dos mitades

1. El asistente

Un programa en Python que:

- busca en los documentos del curso,
- le pide al modelo una respuesta,
- verifica con Isabelle antes de mostrarla.

2. Una formalización propia

- un tema nuevo, no visto en clase,
- al menos 8 ejercicios en Isabelle,
- enunciados y soluciones.

Se califican por separado.

Grupos: de a tres, pero todos saben todo

- Grupos de máximo tres estudiantes.
- Cada grupo, un tema de formalización distinto.

Grupos: de a tres, pero todos saben todo

- Grupos de máximo tres estudiantes.
- Cada grupo, un tema de formalización distinto.
- Cada integrante debe poder correr, explicar y modificar todo el asistente.
- En la sustentación, cualquiera responde por cualquier pieza.

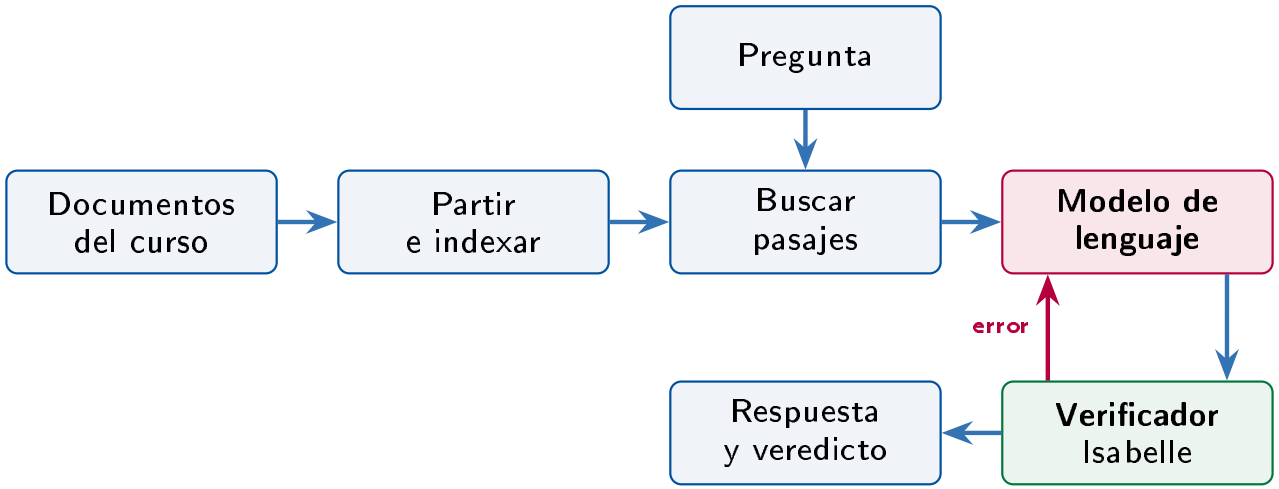
Grupos: de a tres, pero todos saben todo

- Grupos de máximo tres estudiantes.
- Cada grupo, un tema de formalización distinto.
- Cada integrante debe poder correr, explicar y modificar todo el asistente.
- En la sustentación, cualquiera responde por cualquier pieza.

Sugerencia de reparto

Cada uno es responsable principal de una pieza (buscar, verificar, evaluar) y revisa las otras dos.

La arquitectura: el dibujo completo



Hoy construimos solo la caja roja: el modelo.

La flecha roja

1. El modelo propone una demostración.

La flecha roja

1. El modelo propone una demostración.
2. Isabelle la revisa.

La flecha roja

1. El modelo propone una demostración.
2. Isabelle la revisa.
3. Si falla, el mensaje de error vuelve al modelo: «falló aquí, por esto».

La flecha roja

1. El modelo propone una demostración.
2. Isabelle la revisa.
3. Si falla, el mensaje de error vuelve al modelo: «falló aquí, por esto».
4. El modelo corrige y lo intenta otra vez (hasta 5 veces).

La flecha roja

1. El modelo propone una demostración.
2. Isabelle la revisa.
3. Si falla, el mensaje de error vuelve al modelo: «falló aquí, por esto».
4. El modelo corrige y lo intenta otra vez (hasta 5 veces).

Como un estudiante con un profesor que le devuelve el parcial marcado: no le dice la respuesta, le dice dónde se equivocó. Esto lo construimos el jueves 3.

Lo que debe hacer el asistente (1/2)

R1 **Responder con fuentes.** Citar documento y sección, página o lema.

Lo que debe hacer el asistente (1/2)

R1 **Responder con fuentes.** Citar documento y sección, página o lema.

R2 **Admitir lo que no sabe.** Una cita inventada es el peor error posible.

Lo que debe hacer el asistente (1/2)

R1 **Responder con fuentes.** Citar documento y sección, página o lema.

R2 **Admitir lo que no sabe.** Una cita inventada es el peor error posible.

R3 **Demostrar con verificación.** Nunca presentar como correcta una prueba que Isabelle no aceptó.

Lo que debe hacer el asistente (2/2)

R4 **Respetar el estilo.** Si piden deducción natural: `rule`, `erule`, `assumption`...; nada de `auto`, `simp`, `blast`.

Lo que debe hacer el asistente (2/2)

R4 **Respetar el estilo.** Si piden deducción natural: `rule`, `erule`, `assumption`...; nada de `auto`, `simp`, `blast`.

R5 **Dejar registro** de todo. Sin registros no se califica. *(hoy)*

Lo que debe hacer el asistente (2/2)

R4 **Respetar el estilo.** Si piden deducción natural: `rule`, `erule`, `assumption`...; nada de `auto`, `simp`, `blast`.

R5 **Dejar registro** de todo. Sin registros no se califica. *(hoy)*

R6 **(Avanzado) Refutar antes de demostrar** si el enunciado es falso: `quickcheck`, `nitpick`.

Cómo se evalúa: tres configuraciones

Se corre el mismo benchmark tres veces, con el mismo modelo:

	Configuración	Se construye
1	solo el modelo	hoy
2	modelo + búsqueda en los documentos	jueves 2
3	modelo + búsqueda + Isabelle (asistente completo)	jueves 3

Cómo se evalúa: tres configuraciones

Se corre el mismo benchmark tres veces, con el mismo modelo:

	Configuración	Se construye
1	solo el modelo	hoy
2	modelo + búsqueda en los documentos	jueves 2
3	modelo + búsqueda + Isabelle (asistente completo)	jueves 3

Comparar 1, 2 y 3 dice qué aporta cada pieza.

Se califica la mejora, no el número

- Un grupo con un modelo pequeño y gratuito no va a sacar el mismo porcentaje que uno con un modelo de pago.

Se califica la mejora, no el número

- Un grupo con un modelo pequeño y gratuito no va a sacar el mismo porcentaje que uno con un modelo de pago.
- Por eso se califica cuánto mejora de la configuración 1 a la 3, con el mismo modelo.

Se califica la mejora, no el número

- Un grupo con un modelo pequeño y gratuito no va a sacar el mismo porcentaje que uno con un modelo de pago.
- Por eso se califica cuánto mejora de la configuración 1 a la 3, con el mismo modelo.

Y por encima de todo, la honestidad

Un asistente que resuelve el 30% y nunca miente sobre el veredicto vale más que uno que resuelve el 60% y a veces presenta pruebas sin verificar.

Dos benchmarks, dos exámenes

Público

- Los 58 ejercicios de la relación 1.
- Lo tienen desde hoy.
- Las soluciones están en el libro de 2018.

Libro abierto, con el solucionario dentro.

Privado

- Se arma al final.
- Parte II + ejercicios de los *otros* grupos.
- Soluciones fuera del corpus.

Libro abierto, con problemas nuevos.

La pregunta del profesor Fabián

Si el mejor modelo cuesta una suscripción,

¿quién puede hacer este proyecto?

Respuesta: todos. Nada de lo que sigue exige pagar.

Qué es una API

- Cuando usan un chatbot en el navegador, escriben y leen.

Qué es una API

- Cuando usan un chatbot en el navegador, escriben y leen.
- Una API es la misma conversación, pero hecha *desde un programa*: Python envía un mensaje y recibe la respuesta como texto.

Qué es una API

- Cuando usan un chatbot en el navegador, escriben y leen.
- Una API es la misma conversación, pero hecha *desde un programa*: Python envía un mensaje y recibe la respuesta como texto.
- Así el asistente puede preguntar 58 veces seguidas sin que nadie copie y pegue.

Primero, contar llamadas

Configuración	Llamadas al modelo
1. Solo el modelo	58
2. Con búsqueda	58
3. Completa (hasta 5 intentos)	58 a 290
Una corrida	**hasta $\approx$ 400**

Primero, contar llamadas

Configuración	Llamadas al modelo
1. Solo el modelo	58
2. Con búsqueda	58
3. Completa (hasta 5 intentos)	58 a 290
Una corrida	**hasta $\approx$ 400**

Y se corre muchas veces mientras se depura.

El obstáculo real: el límite diario

- Los servicios gratuitos por internet limitan cuántas preguntas se hacen al día.
- OpenRouter sin pagar: unas 50 al día, por cuenta.

El obstáculo real: el límite diario

- Los servicios gratuitos por internet limitan cuántas preguntas se hacen al día.
- OpenRouter sin pagar: unas 50 al día, por cuenta.
- 50 al día y 400 por corrida: ocho días para una sola corrida.

El obstáculo real: el límite diario

- Los servicios gratuitos por internet limitan cuántas preguntas se hacen al día.
- OpenRouter sin pagar: unas 50 al día, por cuenta.
- 50 al día y 400 por corrida: ocho días para una sola corrida.

Conclusión

El camino principal es un modelo *en su propio computador*: sin límite.

Camino A: Gemma 3 en su computador, con Ollama

- **Gemma 3 4B**: modelo abierto de Google, el que propuso el profesor Fabián.
- **Ollama**: programa gratuito que descarga y corre modelos en su máquina.

Camino A: Gemma 3 en su computador, con Ollama

- **Gemma 3 4B**: modelo abierto de Google, el que propuso el profesor Fabián.
- **Ollama**: programa gratuito que descarga y corre modelos en su máquina.
- Funciona en un portátil de 8 GB de RAM, sin tarjeta gráfica (lento, pero funciona).

Camino A: Gemma 3 en su computador, con Ollama

- **Gemma 3 4B**: modelo abierto de Google, el que propuso el profesor Fabián.
- **Ollama**: programa gratuito que descarga y corre modelos en su máquina.
- Funciona en un portátil de 8 GB de RAM, sin tarjeta gráfica (lento, pero funciona).
- Sin límite, sin llave, sin costo, sin internet.

Instalarlo: tres comandos

1. Descargar Ollama de ollama.com e instalarlo.
2. Descargar el modelo (una sola vez, unos 3 GB):

```
ollama pull gemma3:4b
```

3. Probarlo:

```
ollama run gemma3:4b "Que es modus ponens?"
```

Si responde algo, ya tienen un modelo de lenguaje propio.

Camino B: OpenRouter

- Un sitio con muchos modelos; algunos gratuitos (terminan en `:free`), varios mucho más grandes que Gemma 4B.
- Se necesita crear una cuenta y una *llave* (una contraseña para programas).

Camino B: OpenRouter

- Un sitio con muchos modelos; algunos gratuitos (terminan en `:free`), varios mucho más grandes que Gemma 4B.
- Se necesita crear una cuenta y una *llave* (una contraseña para programas).
- Útil si su computador no aguanta el modelo local, o para comparar un modelo pequeño con uno grande.

Camino B: OpenRouter

- Un sitio con muchos modelos; algunos gratuitos (terminan en `:free`), varios mucho más grandes que Gemma 4B.
- Se necesita crear una cuenta y una *llave* (una contraseña para programas).
- Útil si su computador no aguanta el modelo local, o para comparar un modelo pequeño con uno grande.

Lo mejor

Ollama y OpenRouter hablan el mismo idioma: el código cambia de uno a otro con una línea.

Gemma 3 o algo parecido: cada grupo elige

Modelo	Tamaño	Por qué considerarlo
Gemma 3 4B	4B	La referencia; equilibrado, en español
Qwen2.5-Coder 7B	7B	Sabe de código; pide unos 16 GB de RAM
Qwen3 4B	4B	Mismo tamaño, bueno razonando
Phi-4-mini	3,8B	Liviano, bueno en matemáticas
Llama 3.2 3B	3B	Para computadores modestos

Regla: las tres configuraciones, con el mismo modelo.

Qué esperar de un modelo pequeño

- Gemma 4B sabe poco de Isabelle.
- En la configuración 1 va a acertar muy poco.

Qué esperar de un modelo pequeño

- Gemma 4B sabe poco de Isabelle.
- En la configuración 1 va a acertar muy poco.
- **Eso está bien.** Es el punto de partida.

Qué esperar de un modelo pequeño

- Gemma 4B sabe poco de Isabelle.
- En la configuración 1 va a acertar muy poco.
- **Eso está bien.** Es el punto de partida.

La apuesta del proyecto

El verificador iguala: un modelo modesto que se equivoca, lee el error y corrige, puede superar a uno grande que nunca se revisa.

La carpeta Recursos/

Archivo	Para qué
`verificador.py`	le pregunta a Isabelle si algo es correcto
`benchmark_R1.json`	las 58 preguntas públicas
`evaluar.py`	califica sus respuestas (el mismo del profesor)
`ejemplo_respuestas.json`	el formato de respuestas
`README.md`	instrucciones: léanlo

Todo en Python puro: no hay que instalar nada más.

verificador.py: preguntarle a Isabelle

```
from verificador import verificar

r = verificar('lemma "p \\<and> q \\<Longrightarrow> q \\<and> p" by auto')
print(r.ok)           # True si Isabelle la acepta
print(r.segundos)     # cuanto tardo (unos 4 o 5)
print(r.errores)      # lista de errores; vacia si r.ok
```

Si `isabelle` no está en el PATH: `export ISABELLE=/ruta/a/Isabelle2025-2/bin/isabelle`

Lo que `verificar` hace por ustedes

- Acepta una teoría completa o solo un pedazo.

Lo que `verificar` hace por ustedes

- Acepta una teoría completa o solo un pedazo.
- Convierte los símbolos: los modelos escriben $\wedge$, $\longrightarrow$; Isabelle exige `\<and>`, `\<longrightarrow>`.

Lo que `verificar` hace por ustedes

- Acepta una teoría completa o solo un pedazo.
- Convierte los símbolos: los modelos escriben $\wedge$, $\longrightarrow$; Isabelle exige `\<and>`, `\<longrightarrow>`.
- Limpia los errores y dice la línea *del texto que ustedes enviaron*.

Lo que `verificar` hace por ustedes

- Acepta una teoría completa o solo un pedazo.
- Convierte los símbolos: los modelos escriben $\wedge$, $\longrightarrow$; Isabelle exige `\<and>`, `\<longrightarrow>`.
- Limpia los errores y dice la línea *del texto que ustedes enviaron*.
- Corta las pruebas que no terminan.
- `sorry` no pasa.

El benchmark: cómo es una pregunta

```
{
  "id":         "R1/ejercicio_1",
  "estilo":     "dn",
  "enunciado": "lemma ejercicio_1: assumes 1: ... and 2: ... shows \"q\"",
  "contexto":   "theory R1_... imports Main begin ... lemma notnotI ..."
}
```

- `estilo`: `"dn"` (deducción natural) o `"libre"`.
- `contexto`: todo lo que va *antes* del enunciado.

Qué deben responder

Solo lo que va después del enunciado:

```
{
 "R1/ejercicio_1": "proof -\n  show \"q\" using 1 2 by (rule mp)\nqed",
 "R1/ejercicio_2": "using assms by auto"
}
```

Qué deben responder

Solo lo que va después del enunciado:

```
{
 "R1/ejercicio_1": "proof -\n  show \"q\" using 1 2 by (rule mp)\nqed",
 "R1/ejercicio_2": "using assms by auto"
}
```

El evaluador pega las piezas:

$$\texttt{contexto} + \texttt{enunciado} + \text{su respuesta} + \texttt{end}$$

Si el modelo repite el enunciado, la teoría queda rota.

evaluar.py: la calificación

```
python3 evaluar.py benchmark_R1.json respuestas.json --paralelo 4 --informe
    informe.json
```

`aceptada`	correcta y con el estilo pedido
`estilo`	correcta, pero usa `auto` donde se pedía deducción natural
`rechazada`	Isabelle no la acepta
`trampa`	añade lemas, axiomas o `sorry`
`sin respuesta`	falta

Las 58, con 4 en paralelo: unos dos minutos.

El plan del taller: cuatro pasos

1. **Hablar con el modelo** desde Python.
2. **Limpiar** su respuesta y **guardar** todo.
3. **Armar la pregunta** (el *prompt*).
4. **Correr las 58** y calificarlas.

Todo está en el repositorio, carpeta `semana1/taller/`: `git clone https://github.com/cmorregof/repo_proyecto_final.git`

Paso 1a: la configuración

```python
# modelo.py
import json, os, urllib.request

CONFIG = {
    "proveedor":   "ollama",       # o "openrouter"
    "modelo":      "gemma3:4b",    # o el nombre de otro modelo
    "temperatura": 0.2,            # baja = respuestas mas estables
}

URLS = {
    "ollama":     "http://localhost:11434/v1/chat/completions",
    "openrouter": "https://openrouter.ai/api/v1/chat/completions",
}
```

Cambiar de modelo = cambiar `CONFIG`. Nada más.

Paso 1b: la función que pregunta

```
def preguntar(mensajes, cfg=CONFIG):
    cabeceras = {"Content-Type": "application/json"}
    if cfg["proveedor"] == "openrouter":                    # solo aqui hay
     llave
        cabeceras["Authorization"] = "Bearer " + os.environ["OPENROUTER_API_KEY
     "]

    cuerpo = json.dumps({"model": cfg["modelo"],
                         "messages": mensajes,
                         "temperature": cfg["temperatura"]}).encode("utf-8")

    req = urllib.request.Request(URLS[cfg["proveedor"]], data=cuerpo, headers=
     cabeceras)
    with urllib.request.urlopen(req, timeout=300) as r:
        return json.load(r)["choices"][0]["message"]["content"]
```

Entra una lista de mensajes; sale un texto.

Paso 1c: los mensajes tienen roles

```python
from modelo import preguntar

respuesta = preguntar([
    {"role": "system", "content": "Eres un experto en Isabelle/Isar."},
    {"role": "user",   "content": "Que dice la regla mp?"},
])
print(respuesta)
```

- `system`: quién es el modelo, sus reglas.
- `user`: la pregunta.
- `assistant`: lo que el modelo ya respondió (lo usaremos el jueves 3).

Lo que respondió Gemma (en el ensayo)

Explica bien qué es modus ponens... y luego da «un ejemplo en Isabelle/Isar»:

```
module Main
  >:: (P => Q) |- P => Q
  assert (P => Q)
  assert (P)
  assert (Q)
  check
```

Lo que respondió Gemma (en el ensayo)

Explica bien qué es modus ponens... y luego da «un ejemplo en Isabelle/Isar»:

```
module Main
  >:: (P => Q) |- P => Q
  assert (P => Q)
  assert (P)
  assert (Q)
  check
```

- Nada de eso existe en Isabelle: ni `module`, ni `assert`, ni `check`.
- Dice que hay reglas `and` y `or`. Las reales: `conjI`, `conjE`, `disjI1`, `disjE`.

Lo que respondió Gemma (en el ensayo)

Explica bien qué es modus ponens... y luego da «un ejemplo en Isabelle/Isar»:

```
module Main
  >:: (P => Q) |- P => Q
  assert (P => Q)
  assert (P)
  assert (Q)
  check
```

- Nada de eso existe en Isabelle: ni `module`, ni `assert`, ni `check`.
- Dice que hay reglas `and` y `or`. Las reales: `conjI`, `conjE`, `disjI1`, `disjE`.

La mitad es verdad; la otra mitad, Isabelle imaginario.

Ejercicio (3 minutos)

Todos

1. Corran el ejemplo anterior.
2. Pregúntenle: «¿En qué se diferencia `mp` de `impI`?»
3. Cambien `"modelo"` (u `"proveedor"`) y repitan.

¿Alguno inventó una regla que no existe?

Paso 2a: limpiar la respuesta

Los modelos rodean la prueba de explicaciones y comillas triples:

````
Claro! Aqui esta la demostracion:
```isabelle
proof -
  show "q" using 1 2 by (rule mp)
qed
```
Espero que te sirva.
````

Paso 2a: limpiar la respuesta

Los modelos rodean la prueba de explicaciones y comillas triples:

````
Claro! Aqui esta la demostracion:
```isabelle
proof -
  show "q" using 1 2 by (rule mp)
qed
```
Espero que te sirva.
````

```python
# utilidades.py
import re

def extraer_prueba(texto):
    m = re.search(r"```(?:isabelle|isar)?\s*\n(.*?)```", texto, re.S)
    return (m.group(1) if m else texto).strip()
```

Paso 2b: guardar todo (R5)

```python
# utilidades.py  (mismo archivo)
import json, time

def registrar(archivo, **datos):
    datos["hora"] = time.strftime("%Y-%m-%d %H:%M:%S")
    with open(archivo, "a", encoding="utf-8") as f:     # "a" = agregar
        f.write(json.dumps(datos, ensure_ascii=False) + "\n")
```

- Una línea por evento (formato JSON Lines).
- Modo "a": se agrega, nunca se borra.
- Es la memoria del experimento, y la prueba de que el asistente hizo el trabajo.

Paso 3a: las instrucciones fijas

```
# prompt.py
SISTEMA = ("Eres un experto en Isabelle/Isar. Responde SOLO con la "
           "demostracion que va despues del enunciado, sin repetirlo.")

def instruccion_estilo(p):
    if p["estilo"] == "dn":
        return ("Usa solo deduccion natural: rule, erule, assumption, '..'. "
                "Prohibido auto, simp, blast y similares.")
    return "Puedes usar cualquier metodo."
```

«Sin repetirlo»: recuerden que el evaluador ya pone el enunciado.

Paso 3b: armar la pregunta

```python
# prompt.py  (mismo archivo)
def pedir_mensajes(p, extra=""):
    usuario = (f"Contexto de la teoria:\n{p['contexto']}\n\n"
               f"{extra}"
               f"Enunciado:\n{p['enunciado']}\n\n"
               f"{instruccion_estilo(p)}")
    return [{"role": "system", "content": SISTEMA},
            {"role": "user",   "content": usuario}]
```

¿Y extra?

Hoy queda vacío. El jueves 2 ahí entran los pasajes que encuentre la búsqueda: la configuración 2 reutiliza todo esto.

Paso 4a: correr las 58

```python
# config1.py  ---  solo el modelo
import json
from modelo import preguntar, CONFIG
from utilidades import extraer_prueba, registrar
from prompt import pedir_mensajes

preguntas = json.load(open("../../Recursos/benchmark_R1.json", encoding="utf-8"))
respuestas = {}

for p in preguntas:
    msgs   = pedir_mensajes(p)
    crudo  = preguntar(msgs)
    prueba = extraer_prueba(crudo)
    respuestas[p["id"]] = prueba
    registrar("registros/config1.jsonl", id=p["id"], modelo=CONFIG["modelo"],
              prompt=msgs, crudo=crudo, prueba=prueba)

json.dump(respuestas, open("respuestas_config1.json", "w", encoding="utf-8"),
          ensure_ascii=False, indent=1)
```

Creen antes la carpeta `registros/`. Para probar, usen `preguntas[:3]`.

Paso 4b: calificar

```
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config1.json --paralelo 4 --informe informe_c1.json
```

Paso 4b: calificar

```
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config1.json --paralelo 4 --informe informe_c1.json
```

Lo que vimos en el ensayo con Gemma 3 4B (58 preguntas):

aceptada	estilo	rechazada	trampa
0	0	14	44

Paso 4b: calificar

```
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config1.json --paralelo 4 --informe informe_c1.json
```

Lo que vimos en el ensayo con Gemma 3 4B (58 preguntas):

aceptada	estilo	rechazada	trampa
0	0	14	44

¿44 trampas? ¿El modelo hace trampa?

No es trampa: es desobediencia

- Le dijimos «sin repetir el enunciado».
- Gemma lo repite igual: copia el `lemma`, a veces con `oops`.

No es trampa: es desobediencia

- Le dijimos «sin repetir el enunciado».
- Gemma lo repite igual: copia el `lemma`, a veces con `oops`.
- El evaluador ve un `lemma` nuevo en la respuesta y lo trata como intento de colar un lema: ni lo manda a Isabelle.

No es trampa: es desobediencia

- Le dijimos «sin repetir el enunciado».
- Gemma lo repite igual: copia el `lemma`, a veces con `oops`.
- El evaluador ve un `lemma` nuevo en la respuesta y lo trata como intento de colar un lema: ni lo manda a Isabelle.

La lección

Una instrucción en el prompt no es una garantía. Lo que el modelo *debe* cumplir, lo verifica el programa, no la buena fe del modelo.

El arreglo: limpiar mejor

```python
# utilidades.py
INICIO = re.compile(r"^\s*(proof|by|using|apply|from|unfolding)\b", re.M)

def extraer_prueba(texto):
    m = re.search(r"```(?:isabelle|isar)?\s*\n(.*?)```", texto, re.S)
    t = (m.group(1) if m else texto).strip()
    if re.match(r"\s*(lemma|theorem)\b", t):   # repitio el enunciado
        k = INICIO.search(t)
        t = t[k.start():] if k else ""
    return re.sub(r"\boops\s*$", "", t).strip()  # y cierra con oops
```

Ahora esas respuestas sí llegan a Isabelle, y aparecen los errores de verdad. Ese es el número que vale como línea base.

Errores reales de Gemma (verificados con Isabelle)

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`. Isabelle: `Undefined fact`.

Errores reales de Gemma (verificados con Isabelle)

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`. Isabelle: `Undefined fact`.
- **Ejercicio 12:** inventa la regla `imply`.

Errores reales de Gemma (verificados con Isabelle)

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`.
  Isabelle: `Undefined fact`.
- **Ejercicio 12:** inventa la regla `imply`.
- **Ejercicio 1:** `using 1, 2 by rule` falla solo por la coma; sin ella, Isabelle la acepta.

Errores reales de Gemma (verificados con Isabelle)

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`.
  Isabelle: `Undefined fact`.
- **Ejercicio 12:** inventa la regla `imply`.
- **Ejercicio 1:** `using 1, 2 by rule` falla solo por la coma; sin ella, Isabelle la acepta.
- **Pregunta libre:** «MP es un *Modo de Pregunta*, una inferencia inductiva», con enlaces que no existen.

Errores reales de Gemma (verificados con Isabelle)

- **Ejercicio 11:** inventa la regla `implicacion_con_consecuencia`. Isabelle: `Undefined fact`.
- **Ejercicio 12:** inventa la regla `imply`.
- **Ejercicio 1:** `using 1, 2 by rule` falla solo por la coma; sin ella, Isabelle la acepta.
- **Pregunta libre:** «MP es un *Modo de Pregunta*, una inferencia inductiva», con enlaces que no existen.

Ese número, con la limpieza corregida, es la línea base. Anótenlo.

Para pensar

La configuración 1 no verifica nada.

Ninguna de esas respuestas se le puede mostrar a un usuario como correcta.

Por eso es la línea base, y no el asistente.

Temas de formalización (1/2)

Rama	Teorema central	Nivel
Codificación	`decodificar (codificar xs) = xs`	$\star$
Criptografía clásica	César o Vigenère: descifrar invierte a cifrar	$\star$
Algoritmos	Inserción: ordena y es permutación	$\star\star$
Estructuras de datos	Árboles de búsqueda: la inserción preserva el orden	$\star\star$
Compiladores	Expresiones a máquina de pila	$\star\star$
Lenguajes formales	Palíndromos o paréntesis balanceados	$\star\star$

Temas de formalización (2/2)

Rama	Teorema central	Nivel
Lógica computacional	La forma normal negativa preserva el significado	⋆⋆
Arquitectura	Sumador binario sobre listas de bits	⋆⋆⋆
Grafos	Alcanzabilidad como cierre reflexivo-transitivo	⋆⋆⋆
Lenguajes de programación	Tipado de expresiones: preservación	⋆⋆⋆
Teoría de la computación	Autómatas: producto e intersección	⋆⋆⋆
Software (entusiastas)	Verificar un programa imperativo	⋆⋆⋆

Un tema por grupo, sin repetir. Si se agotan: matemáticas.

Para el próximo jueves

Tarea

1. Configuración 1 completa sobre las 58: respuestas, registro e informe.

Para el próximo jueves

Tarea

1. Configuración 1 completa sobre las 58: respuestas, registro e informe.
2. Leer diez registros y clasificar los errores (sintaxis, reglas inventadas, orden, `auto` prohibido…).

Para el próximo jueves

Tarea

1. Configuración 1 completa sobre las 58: respuestas, registro e informe.
2. Leer diez registros y clasificar los errores (sintaxis, reglas inventadas, orden, `auto` prohibido...).
3. Tema elegido (tres opciones en orden) y dos enunciados escritos en Isabelle.

Próximo jueves: la biblioteca. Buscar en los documentos y citar sin inventar.

La verdad no la pone el modelo más caro:
la pone Isabelle.